

Simulation-Based Inference

Creating pseudo-randomness with computers

Paul Bürkner

What is randomness?

Pseudo random number generators

Informal definition: Algorithms that produce sequences whose next value is unpredictable from the previous values

They *appear random* to the unknown eye

Essentially, we need chaotic functions

Computers are discrete, so we need chaotic functions over integers

Discrete uniform distributions

Every discrete density of the form

$$f(x \mid N) = \left\{ \begin{array}{ll} \frac{1}{N}, & x = x_1, x_2, \dots, x_N, \\ 0, & \text{otherwise} \end{array} \right\},$$

with $N \in \mathbb{N}$, defines a density of the discrete uniform distribution with support points $x_1, x_2, \dots, x_N$

To generate a discrete uniform distribution on the numbers $x_1, \dots, x_N$, the interval $[0, \text{maxinteger}]$, is divided into N equal parts, each of which is assigned to an x_i .

For the Definition of Generators First, let us recall the modulo operation:

Let $a, b \in \mathbb{Z}$ and $m \in \mathbb{N}$. Then the equivalence relation

$$a \equiv b :\Leftrightarrow a - b \text{ is divisible by } m$$

partitions the set of integers $\mathbb{Z}$ into so-called residue classes modulo m . We write $a = \text{mod}_m(b)$ and choose as representatives of the residue class modulo m the number symbols $0, 1, 2, \dots, m - 1$. The set of representatives is denoted by $\mathbb{Z}_m$.

Linear Congruential Sequences (LCS)

Modulo Operators form the basis for many common random number generators

A Linear Congruential Sequence $\{x_j\}_{j \in \mathbb{N}}$ is recursively defined by:

$$x_{j+1} := \text{mod}_m(ax_j + c), \text{ where}$$

$x_0 \in \mathbb{N}$ is the initial value,

$a \in \mathbb{N}$ is the multiplier,

$c \in \mathbb{N}$ is the increment,

$m \in \mathbb{N}$ is the modulus and $m > x_0, a, c$.

1. Let $a := 5$, $c := 7$, $x_0 := 4$, $m := 13$.

Then we have:

$$x_1 = \text{mod}_{13}(27) = 1, \quad x_2 = 12, \quad x_3 = 2, \quad x_4 = 4 = x_0, \\ x_5 = x_1 \dots$$

2. Let $a := 5$, $c := 7$, $x_0 := 4$, $m := 8$.

Then we have:

$$x_1 = 3, \quad x_2 = 6, \quad x_3 = 5, \quad x_4 = 0, \\ x_5 = 7, \quad x_6 = 2, \quad x_7 = 1, \quad x_8 = 4 = x_0, \\ x_9 = x_1 \quad \dots$$

In both examples, the number sequences behave cyclically. This motivates the following definition:

A number sequence $\{x_j\}_{j \in \mathbb{N}}$ has a period $\mu \in \mathbb{N}$ if $x_j = x_{j+\mu}$ for all j .

Theorem: Every LCS has a period $\leq m$.

Proof: We have: $x_{j+1} := \text{mod}_m(ax_j + c) < m$ for all $j \in \mathbb{N}_0$. Therefore, among the first $m + 1$ sequence elements, at least two must be equal, for instance x_λ and $x_{\lambda+\mu}$ with $1 \leq \mu \leq m$. Since the equation above uniquely defines the successor x_{j+1} for x_j , it follows that $x_{j+\mu} = x_j, \forall j \geq \lambda$. Thus, the period is $\leq \mu \leq m$. $\square$

Choosing the modulo m

Choose m very large

- Leads to longer periods

Choose m to be a power of 2, say, $m = 2^{31}$ or $m = 2^{32}$

- Enables efficient integer division in the binary system via elimination of the last bits

Multiply-With-Carry (MWC) generators

Here we want to present a simple extension of the LCS that is competitive in modern generator tests and easy to implement.

The class of *multiply-with-carry* (MWC) generators was proposed by Marsaglia (1996).

It generalizes the LCS in that it uses variable increments:

$$x_{j+1} = \text{mod}_m(ax_j + c_j) \quad \text{and} \quad c_{j+1} = \left\lfloor \frac{ax_j + c_j}{m} \right\rfloor. \quad (1)$$

The only difference to LCS is that the increment retains the multiple of the modulo operation ("carries").

For $a = 698769069$ and $m = 2^{32}$, we can obtain periods of about $2^{50.4} \approx 10^{18.2}$.

Common random number generator developed by Makoto Matsumoto and Takuji Nishimura (1997):

(1) Define $N = 624$ initial values $Y_1, \dots, Y_N$: the seed consists of 624 numbers.

(2) Calculate the next sequence element Y_i as:

$$Y_i = Y_{i-227} \oplus \left\lfloor \frac{h}{2} \right\rfloor \oplus (\text{mod}_2(h) \cdot 9908B0DF_{16}) \text{ where} \\ h = Y_{i-N} - \text{mod}_m(Y_{i-N}) + \text{mod}_m(Y_{i-N-1}) \text{ with } m = 2^{31}.$$

$\oplus$ represents the bitwise XOR operation of the binary representations of two integer numbers: returns 1 if the compared bits are different and 0 if they are the same.

(continued on the next page)

(3) Transform Y_i to obtain the random number Z_i to be used:

$$x = Y_i \oplus \lfloor \frac{Y_i}{2^{11}} \rfloor$$

$$y = x \oplus ((x \cdot 2^7) \wedge 9D2C5680_{16})$$

$$z = y \oplus ((y \cdot 2^{15}) \wedge EFC60000_{16})$$

$$Z_i = z \oplus \lfloor \frac{z}{2^{18}} \rfloor$$

$\wedge$ represents the bitwise AND operation of the binary representations of two integer numbers: returns 1 if both bits are 1 and 0 otherwise.

The period length of the Mersenne Twister is approximately 10^{6001} . The generated random numbers are independent in 623 dimensions. This means: The 624th random number depends on the 1st random number, but not on the random numbers 2 through 623.

How can we test if a random number generator actually creates “random” uniform samples?

Many test are based on the following basic Chi-square test:

If you make M independent observations, which can fall into K different categories, the hypothesis is tested:

H_0 : The probability that an observation falls into the category s is equal to p_s , $1 \leq s \leq K$.

Here M is to be chosen so large that it satisfies

$$Mp_s \geq 5, \quad 1 \leq s \leq K$$

Chi-square test

Let Y_s be number of observations in category s .

To assess the hypothesis, we use the following test statistic:

$$V = \sum_{s=1}^K \frac{(Y_s - Mp_s)^2}{Mp_s}$$

Assuming that the null hypothesis is correct, V approximately chi-square distributed with $K - 1$ degrees of freedom for large enough M .

M should be large enough to fulfill the approximation criterion $Mp_s \geq 5$ but also not too large (for computing time reasons and to avoid very small differences becoming significant), e.g. $M = 10000$.

Example: Uniformity test

k -tuples of consecutive elements of the sequence $\{x_i\}_{i \in \mathbb{N}_0}$ are tested for independence and uniformity

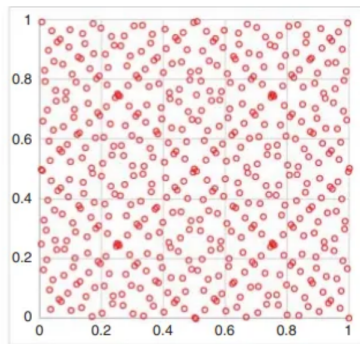
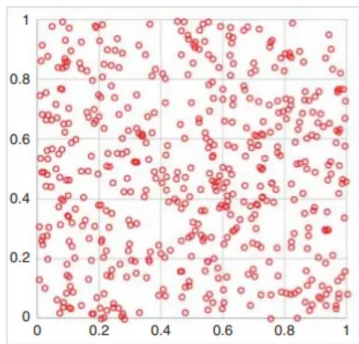
Define $y_i^{(D)} := \lfloor Dx_i/m \rfloor$, for $D \in \mathbb{N}$, $D < m$. It holds that $0 \leq y_i^{(D)} < D$, $y_i^{(D)} \in \mathbb{N}_0$.

We apply the chi-square test for $K := D^k$, $j = 1, 2, \dots$, and probabilities $p_s = \frac{1}{D^k}$ to the tuples:

$$\begin{aligned} k = 1 & \quad \text{on} \quad \left(y_j^{(D)}\right), \\ k = 2 & \quad \text{on} \quad \left(y_{2j-1}^{(D)}, y_{2j}^{(D)}\right) \text{ and} \\ k = 3 & \quad \text{on} \quad \left(y_{3j-2}^{(D)}, y_{3j-1}^{(D)}, y_{3j}^{(D)}\right). \end{aligned}$$

Sobol sequences

Sobol sequences deterministically generate high uniformity



left: random, right: Sobol — Modern Computational Finance (Antoine Savine, Wiley, 2018), page 205